

Regiones Básicas de la Geometría Plana y su Área

Departamento de Matemáticas

6 de agosto de 2026

Resumen

Este documento presenta una descripción concisa de las regiones planas más elementales: el cuadrado, el rectángulo, el triángulo, el círculo y el paralelogramo. Para cada una, se proporciona la fórmula del área, su interpretación geométrica y una representación gráfica mediante el paquete `tikz`. El objetivo es ofrecer una referencia visual y analítica para el cálculo de áreas en geometría básica.

1. Introducción

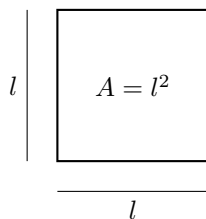
El área de una región plana es una medida de la superficie que ocupa dicha figura en el plano. Las figuras más básicas —cuadrado, rectángulo, triángulo, círculo y paralelogramo— son la base para entender regiones más complejas. En este artículo, se revisan sus propiedades fundamentales y se ilustran con diagramas.

2. Cuadrado

El cuadrado es un polígono regular de cuatro lados iguales y cuatro ángulos rectos. Su área se calcula como el producto de su lado por sí mismo:

$$A = l^2$$

donde l es la longitud del lado.

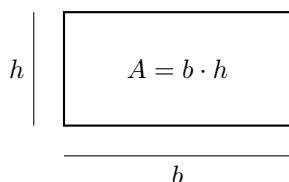


3. Rectángulo

El rectángulo tiene lados opuestos iguales y cuatro ángulos rectos. Su área es el producto de su base por su altura:

$$A = b \cdot h$$

donde b es la base y h la altura.

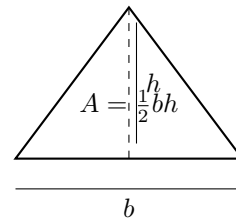


4. Triángulo

El triángulo es un polígono de tres lados. Su área es la mitad del producto de su base por su altura:

$$A = \frac{1}{2} \cdot b \cdot h$$

donde b es la base y h la altura perpendicular a la base.

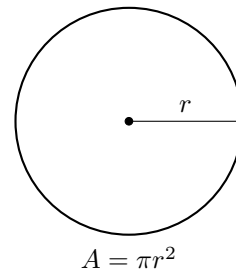


5. Círculo

El círculo es la región limitada por una circunferencia. Su área depende del radio r :

$$A = \pi r^2$$

donde r es el radio.

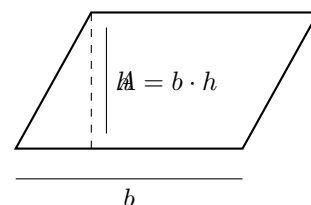


6. Paralelogramo

El paralelogramo es un cuadrilátero con lados opuestos paralelos. Su área es el producto de su base por su altura:

$$A = b \cdot h$$

donde b es la base y h la altura perpendicular a la base.



7. Comparativa y aplicaciones

Estas figuras son la base de la geometría euclidiana. El área del triángulo se puede derivar del rectángulo; el círculo, mediante integración o aproximación por polígonos; y el paralelogramo, del rectángulo mediante corte y reubicación. Estas fórmulas son esenciales en campos como la ingeniería, la arquitectura y la física.

8. Conclusión

Hemos revisado las regiones planas más básicas y sus respectivas fórmulas de área. La representación gráfica facilita la comprensión de las variables involucradas. Estos conceptos sientan las bases para el estudio de áreas de figuras compuestas y sólidos en el espacio.

Regiones Básicas de la Geometría Plana

Cálculo de Áreas

Prof. Juan Carlos Pérez

Matemáticas Aplicadas

6 de agosto de 2026

Introducción

La geometría plana estudia las figuras que pueden dibujarse en un plano. El **área** es una medida de la superficie que ocupa una región cerrada. En este artículo revisamos las figuras más elementales, sus propiedades y las fórmulas para calcular su área, deduciéndolas a partir de conceptos intuitivos como el conteo de cuadrados unitarios o la descomposición en figuras conocidas.

El Cuadrado

El cuadrado es un polígono regular de cuatro lados iguales y cuatro ángulos rectos. Si su lado mide l , su área se obtiene multiplicando base por altura, que en este caso coinciden:

$$A = l \cdot l = l^2$$

Ejemplo: Un cuadrado de lado 5 cm tiene área 25 cm^2 .

El Rectángulo

Es un paralelogramo con cuatro ángulos rectos. Si su base mide b y su altura h , el área es:

$$A = b \cdot h$$

Idea: Se puede dividir en $b \times h$ cuadrados unitarios.

El Triángulo

El triángulo es la región más simple con tres lados. Su área es la mitad del producto de una base por su altura correspondiente:

$$A = \frac{1}{2} \cdot b \cdot h$$

Deducción: Un triángulo es exactamente la mitad de un rectángulo (o paralelogramo) de igual base y altura.

Triángulo rectángulo

Si los catetos son a y b , el área es $\frac{1}{2}ab$.

Triángulo equilátero

Si el lado es l , la altura es $h = \frac{\sqrt{3}}{2}l$, luego:

$$A = \frac{\sqrt{3}}{4}l^2$$

El Paralelogramo

Un paralelogramo tiene lados opuestos paralelos e iguales. Su área es base por altura (perpendicular a la base):

$$A = b \cdot h$$

Observación: Un rectángulo es un caso particular, y cualquier paralelogramo puede transformarse en un rectángulo de igual base y altura mediante un corte y reubicación.

El Trapecio

El trapecio tiene dos bases paralelas de longitudes B (mayor) y b (menor), y altura h . Su área es el promedio de las bases multiplicado por la altura:

$$A = \frac{B + b}{2} \cdot h$$

Deducción: Se puede descomponer en un rectángulo y dos triángulos, o duplicar el trapecio para formar un paralelogramo.

El Rombo

El rombo es un paralelogramo con cuatro lados iguales, pero no necesariamente ángulos rectos. Su área se puede calcular como:

$$A = \frac{d_1 \cdot d_2}{2}$$

donde d_1 y d_2 son las longitudes de sus diagonales. *Justificación:* Las diagonales son perpendiculares y dividen al rombo en cuatro triángulos rectángulos congruentes.

El Círculo

El círculo es el conjunto de puntos que están a una distancia menor o igual que el radio r de un punto central. Su área está dada por la famosa fórmula:

$$A = \pi r^2$$

Idea intuitiva: Se puede aproximar dividiendo el círculo en sectores muy delgados y reordenándolos para formar una figura casi rectangular de base πr y altura r .

Tabla Resumen

Figura	Área
Cuadrado	l^2
Rectángulo	$b \cdot h$
Triángulo	$\frac{1}{2}bh$
Paralelogramo	$b \cdot h$
Trapezio	$\frac{B+b}{2}h$
Rombo	$\frac{d_1 d_2}{2}$
Círculo	πr^2

Aplicaciones y Conclusiones

El cálculo de áreas es fundamental en disciplinas como la arquitectura, la ingeniería, la física y la vida cotidiana. Conocer estas fórmulas permite resolver problemas de recubrimiento de superficies, cálculo de terrenos, diseño de objetos y mucho más.

Todas estas fórmulas provienen de principios geométricos sencillos: la descomposición en figuras más simples, la equivalencia por traslación y el concepto de límite (como en el círculo). Dominarlas es el primer paso para adentrarse en la geometría analítica y el cálculo integral.

Regiones Básicas de la Geometría Plana y Cálculo de Áreas

Departamento de Matemáticas

6 de agosto de 2026

Resumen

Este documento presenta una revisión de las figuras geométricas planas más elementales, enfocándose en el cálculo de sus áreas. Se incluyen representaciones gráficas generadas con TikZ, donde la región interna se destaca en color azul cielo. Cada sección describe la figura, su fórmula de área y un ejemplo ilustrativo.

Introducción

La geometría plana estudia las figuras bidimensionales y sus propiedades. Una de las magnitudes más importantes es el área, que mide la extensión de una superficie. A continuación, se describen las figuras básicas: cuadrado, rectángulo, triángulo, círculo y trapecio.

1. Cuadrado

El cuadrado es un polígono regular de cuatro lados iguales y cuatro ángulos rectos.

Fórmula del área

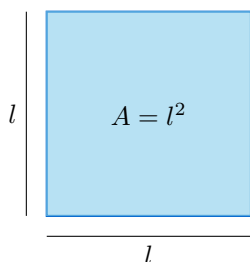
$$A = l^2$$

donde l es la longitud del lado.

Ejemplo

Si $l = 4$ cm, entonces $A = 16$ cm².

Representación gráfica



2. Rectángulo

El rectángulo tiene lados opuestos iguales y cuatro ángulos rectos.

Fórmula del área

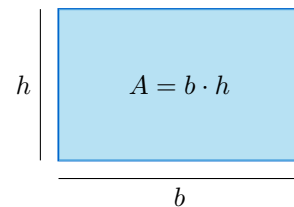
$$A = b \cdot h$$

donde b es la base y h la altura.

Ejemplo

Si $b = 5$ cm y $h = 3$ cm, $A = 15$ cm².

Representación gráfica



3. Triángulo

El triángulo es un polígono de tres lados. El área se calcula a partir de la base y la altura.

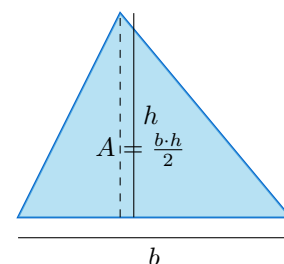
Fórmula del área

$$A = \frac{b \cdot h}{2}$$

Ejemplo

Si $b = 6$ cm y $h = 4$ cm, $A = 12$ cm².

Representación gráfica



4. Círculo

El círculo es la región delimitada por una circunferencia. Su área depende del radio.

Fórmula del área

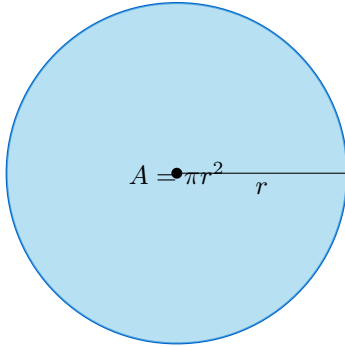
$$A = \pi r^2$$

donde r es el radio.

Ejemplo

Si $r = 3$ cm, $A = 9\pi \approx 28,27$ cm^2 .

Representación gráfica



5. Trapecio

El trapecio es un cuadrilátero con un par de lados paralelos (bases).

Fórmula del área

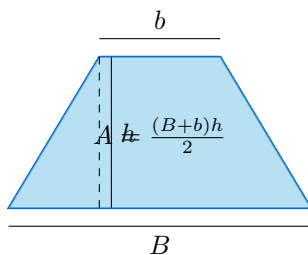
$$A = \frac{(B + b) \cdot h}{2}$$

donde B es la base mayor, b la base menor y h la altura.

Ejemplo

Si $B = 6$ cm, $b = 4$ cm y $h = 3$ cm, $A = 15$ cm^2 .

Representación gráfica



Conclusión

Hemos revisado las figuras planas más fundamentales y sus respectivas fórmulas para el cálculo del área. Estas nociones son la base para el estudio de geometría más avanzada y tienen aplicaciones en diversas áreas de la ciencia y la ingeniería. El uso de representaciones gráficas con colores como el azul cielo facilita la comprensión visual de la región cuyo área se está calculando.

Regiones para el Calculo de Areas

Matematicas Aplicadas

6 de agosto de 2026

Resumen

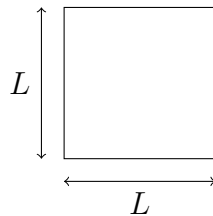
Este documento recopila las principales regiones geometricas planas cuyo calculo de area es fundamental en matematicas, fisica e ingenieria. Se incluyen figuras ilustrativas, formulas y observaciones clave para cada una.

Introduccion

El area de una region plana es una medida de la superficie encerrada por sus fronteras. Conocer las formulas para las figuras basicas permite descomponer regiones mas complejas y resolver problemas de integracion, optimizacion y modelado.

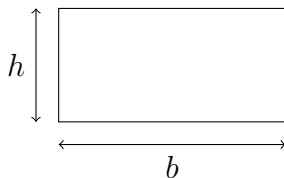
1. Regiones Poligonales

1.1. Cuadrado



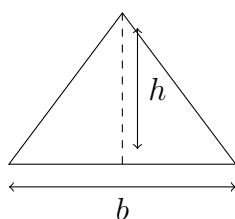
$$A = L^2$$

1.2. Rectangulo



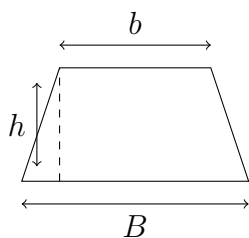
$$A = b \cdot h$$

1.3. Triangulo



$$A = \frac{b \cdot h}{2}$$

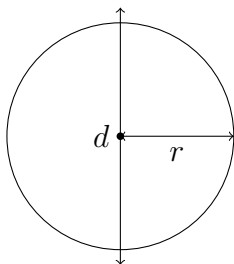
1.4. Trapecio



$$A = \frac{(B + b) \cdot h}{2}$$

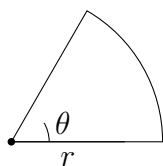
2. Regiones Circulares

2.1. Circulo



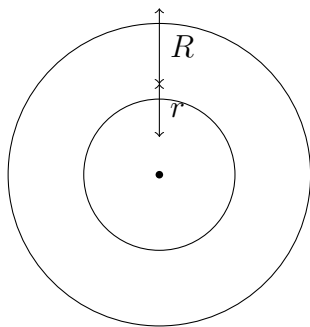
$$A = \pi r^2$$

2.2. Sector Circular



$$A = \frac{\theta}{2} r^2 \quad (\text{radianes})$$

2.3. Corona Circular



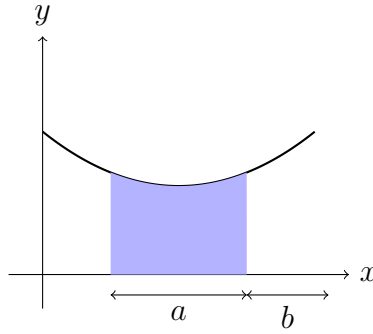
$$A = \pi(R^2 - r^2)$$

3. Regiones bajo Curvas (Calculo Integral)

3.1. Area bajo una curva

Para $f(x) \geq 0$ en $[a, b]$:

$$A = \int_a^b f(x) dx$$



3.2. Area entre dos curvas

Para $f(x) \geq g(x)$ en $[a, b]$:

$$A = \int_a^b (f(x) - g(x)) dx$$

3.3. Area en coordenadas polares

Para $r = r(\theta)$, $\alpha \leq \theta \leq \beta$:

$$A = \frac{1}{2} \int_{\alpha}^{\beta} r(\theta)^2 d\theta$$

4. Regiones Irregulares (Metodos Numericos)

- Regla del Trapecio:

$$A \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(x_i) \right]$$

- Regla de Simpson (1/3):

$$A \approx \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{i \text{ impar}} f(x_i) + 2 \sum_{i \text{ par}} f(x_i) \right]$$

Tabla Resumen

Region	Area
Cuadrado	L^2
Rectangulo	$b \cdot h$
Triangulo	$\frac{b \cdot h}{2}$
Trapecio	$\frac{(B+b)h}{2}$
Circulo	πr^2
Sector circular	$\frac{1}{2} r^2 \theta$
Corona circular	$\pi(R^2 - r^2)$
Bajo curva	$\int_a^b f(x) dx$
Entre curvas	$\int_a^b (f - g) dx$
Polares	$\frac{1}{2} \int_{\alpha}^{\beta} r^2 d\theta$

Conclusion

El conocimiento de estas regiones y sus formulas es esencial para resolver problemas de geometria, fisica, estadistica y calculo. Ademias, permite abordar regiones compuestas mediante descomposicion en figuras simples o mediante integracion.

```
# mi primer cuaderno
# uso del comando print(.)
print ("hola mundo")
```

hola mundo

```
print("mi nombre es: ") #Andrea
```

mi nombre es:

```
# Mi primer programa en Python
print("¡Hola, mundo desde Google Colab!")
print("Python es increíble")
print("¡Estoy aprendiendo a programar!")
```

[Show hidden output](#)

```
# Personaliza tu información
nombre = "andre"
edad = 20
ciudad = "Medellin"
lenguaje_favorito = "Python"
print(f"Hola, soy {nombre}")
print(f"Tengo {edad} años")
print(f"Vivo en {ciudad}")
print(f"Mi lenguaje favorito es {lenguaje_favorito}")
```

Hola, soy andre
Tengo 20 años
Vivo en Medellin
Mi lenguaje favorito es Python

```
print(f"Hola, soy {nombre}. tengo {edad}anos")
```

Hola, soy andre. tengo 20anos

```
print(f"vivo en la ciudad de {ciudad} y programo en {lenguaje_favorito}")
```

vivo en la ciudad de Medellin y programo en Python

```
#Ejercicio 3: Python como calculadora
#Explora las operaciones matemáticas básicas:
# Python como calculadora
# operacion de suma
print("=== Operaciones Básicas ===")
print(f"2 + 3 = {2 + 3}")
```

[Show hidden output](#)

```
# Python como calculadora
# operacion de resta
print("=== Operaciones Básicas ===")
print(f"10 - 4 = {10 - 4}")
```

=== Operaciones Básicas ===
10 - 4 = 6

```
# Python como calculadora
# operacion de multiplicacion
print("=== Operaciones Básicas ===")
print(f"6 * 7 = {6 * 7}")
```

=== Operaciones Básicas ===
6 * 7 = 42

```
# Python como calculadora
# operacion de division
print("=== Operaciones Básicas ===")
```

```
print(f"15 / 3 = {15 / 3}")
```

```
=== Operaciones Básicas ===  
15 / 3 = 5.0
```

```
# Python como calculadora  
# opracion de potenciacion  
print("=== Operaciones Básicas ===")  
print(f"2 ** 8 = {2 ** 8}")
```

```
=== Operaciones Básicas ===  
2 ** 8 = 256
```

```
# Python como calculadora  
# operacion de residuo de division  
print("=== Operaciones Básicas ===")  
print(f"17 % 5 = {17 % 5}")
```

```
=== Operaciones Básicas ===  
17 % 5 = 2
```

```
# Python como calculadora  
# operacion de residuo de division  
print("=== Operaciones Básicas ===")  
print(f"3 * 5 + 17 % 3 = {3 * 5 + 17 % 3}")
```

```
=== Operaciones Básicas ===  
3 * 5 + 17 % 3 = 17
```

```
#Ejercicio 4: Prueba en Visual Studio Code Online  
# Crear un archivo Python en VS Code  
# 1. Haz clic en "Abrir Visual Studio Code" arriba  
# 2. Crea un nuevo archivo: archivo.py  
# 3. Copia cualquiera de los ejercicios anteriores  
# 4. Guarda el archivo y ejecútalo
```

```
print("¡Hola desde Visual Studio Code!")  
nombre = input("¿Cuál es tu nombre? ")  
print(f"¡Bienvenido a Python, {nombre}!")
```

[Show hidden output](#)

```
#ingresar dos #s tal que a % b sea = 2
```

```
print(f"{a} % {b}= {a % b}")
```

```
texto = "hola mundo python"  
  
texto_mayusculas = texto.upper()  
longitud_texto = len(texto)  
  
print(f"Mayúsculas: {texto_mayusculas}")  
print(f"Longitud: {longitud_texto}")
```


Comandos Básicos en Python

input, print y operaciones con variables enteras

Documento elaborado para el curso de Programación

22 de julio de 2026

Índice

1. Introducción	2
2. El comando input()	2
2.1. ¿Qué es input()?	2
2.2. Sintaxis básica	2
2.3. Ejemplos de uso	2
2.4. Importante: input() siempre devuelve texto	3
3. El comando int()	3
3.1. ¿Qué es int()?	3
3.2. Sintaxis básica	3
3.3. Ejemplos de conversión	3
3.4. Combinando input() y int()	3
3.5. Manejo de errores con int()	4
4. El comando print()	4
4.1. ¿Qué es print()?	4
4.2. Sintaxis básica	4
4.3. Parámetros importantes de print()	4
4.3.1. separador (sep)	4
4.3.2. final (end)	4
4.4. Ejemplos de uso de print()	5
5. Operaciones con variables numéricas enteras	5
5.1. Declaración de variables enteras	5
5.2. Operadores aritméticos básicos	5
5.3. Ejemplos de operaciones	5
5.4. Operadores de asignación	6
6. Programas completos de ejemplo	6
6.1. Ejemplo 1: Calculadora simple	6
6.2. Ejemplo 2: Conversión de unidades	7
6.3. Ejemplo 3: Promedio de calificaciones	7
7. Resumen y recomendaciones	7
7.1. Puntos clave	7
7.2. Errores comunes a evitar	8
8. Conclusión	8

1 Introducción

Python es un lenguaje de programación de alto nivel que se caracteriza por su sintaxis clara y legible. En este documento, exploraremos tres elementos fundamentales para cualquier programa en Python:

- El comando `input()` para la entrada de datos
- El comando `print()` para la salida de información
- Las operaciones con variables numéricas enteras
- El comando `int()` para conversión de tipos

Estos elementos constituyen la base para interactuar con el usuario y realizar cálculos básicos.

2 El comando `input()`

2.1 ¿Qué es `input()`?

El comando `input()` en Python permite recibir información del usuario a través del teclado. Su funcionamiento es simple: cuando el programa llega a esta instrucción, se detiene y espera a que el usuario escriba algo y presione Enter.

2.2 Sintaxis básica

La sintaxis básica del comando es:

```
1 variable = input("Mensaje para el usuario: ")
```

El texto dentro de los paréntesis es opcional y sirve como mensaje informativo para el usuario.

2.3 Ejemplos de uso

```
1 # Ejemplo 1: Solicitar un nombre
2 nombre = input("    Cmo    te llamas? ")
3 print("Hola", nombre)
4
5 # Ejemplo 2: Solicitar edad
6 edad = input("    Cuntos    a os tienes? ")
7 print("Tienes", edad, "a os")
8
9 # Ejemplo 3: M ltiples preguntas
10 pais = input(" De    qu    pa s eres? ")
11 ciudad = input(" En    qu    ciudad vives? ")
12 print("Vives en", ciudad + ", ", pais)
```


2.4 Importante: input() siempre devuelve texto

Es fundamental recordar que `input()` siempre devuelve una cadena de texto (*string*), incluso cuando el usuario ingresa números. Por ejemplo:

```
1 numero = input("Ingresa un número: ")
2 print(type(numero)) # Salida: <class 'str'>
```

Esto significa que si intentamos hacer operaciones matemáticas directamente con el valor ingresado, obtendremos errores.

3 El comando int()

3.1 ¿Qué es int()?

El comando `int()` es una función incorporada en Python que convierte un valor a un número entero. Esta función es esencial cuando trabajamos con datos ingresados por el usuario mediante `input()`.

3.2 Sintaxis básica

```
1 numero_entero = int(valor_a_convertir)
```

3.3 Ejemplos de conversión

```
1 # Convertir texto a entero
2 texto = "123"
3 numero = int(texto)
4 print(numero) # 123
5 print(type(numero)) # <class 'int'>
6
7 # Convertir número decimal a entero (trunca la parte decimal)
8 decimal = 3.14
9 entero = int(decimal)
10 print(entero) # 3
11
12 # Convertir input a entero
13 edad_texto = input("Ingresa tu edad: ")
14 edad = int(edad_texto)
15 print("El año que viene tendrás", edad + 1, "años")
```

3.4 Combinando input() y int()

La combinación más común es recibir datos del usuario y convertirlos a enteros para realizar operaciones:

```
1 # Solicitar un número y convertirlo a entero
2 numero1 = int(input("Ingresa el primer número: "))
3 numero2 = int(input("Ingresa el segundo número: "))
```

```

4
5 # Realizar operaciones
6 suma = numero1 + numero2
7 print("La suma es:", suma)

```

3.5 Manejo de errores con int()

Es importante tener en cuenta que `int()` puede generar errores si intentamos convertir un valor que no es numérico:

```

1 # Esto generar un error (ValueError)
2 # numero = int("hola")

```

Para evitar estos errores, podemos usar estructuras de control como `try` y `except`.

4 El comando print()

4.1 ¿Qué es print()?

El comando `print()` es la forma más básica de mostrar información en Python. Permite enviar texto, números y otros tipos de datos a la consola o salida estándar.

4.2 Sintaxis básica

```

1 print("Texto a mostrar")
2 print(variable)
3 print("Texto", variable, "m s texto")

```

4.3 Parámetros importantes de print()

4.3.1. separador (sep)

Permite especificar qué carácter se utiliza para separar los elementos:

```

1 nombre = "Ana"
2 edad = 25
3 print(nombre, edad, sep=" - ")    # Ana - 25
4 print(1, 2, 3, sep=",")          # 1,2,3

```

4.3.2. final (end)

Permite especificar qué carácter se coloca al final de la línea:

```

1 print("Hola", end=" ")
2 print("mundo")    # Hola mundo
3
4 print("Primero", end=", ")
5 print("segundo")  # Primero, segundo

```

4.4 Ejemplos de uso de print()

```
1 # Mostrar texto literal
2 print("Bienvenido al programa")
3
4 # Mostrar variables
5 nombre = "Carlos"
6 edad = 30
7 print("Nombre:", nombre)
8 print("Edad:", edad)
9
10 # Mostrar mltiples elementos
11 print("Nombre:", nombre, "Edad:", edad)
12
13 # Mostrar resultados de operaciones
14 resultado = 15 + 20
15 print("El resultado es:", resultado)
16
17 # Formateo de salida
18 print(f"Nombre: {nombre}, Edad: {edad}")
19 print("Nombre: {}, Edad: {}".format(nombre, edad))
```

5 Operaciones con variables numéricas enteras

5.1 Declaración de variables enteras

En Python, no es necesario declarar el tipo de variable; el intérprete lo infiere automáticamente:

```
1 # Asignaci n directa
2 edad = 25
3 cantidad = 100
4 temperatura = -5
5
6 # Desde input (con conversi n)
7 numero = int(input("Ingresa un n mero: "))
```

5.2 Operadores aritméticos básicos

Python soporta los siguientes operadores para números enteros:

5.3 Ejemplos de operaciones

```
1 # Variables enteras
2 a = 10
3 b = 3
4
5 # Operaciones b sicas
6 suma = a + b
```

Operador	Descripción	Ejemplo
+	Suma	$5 + 3 = 8$
-	Resta	$5 - 3 = 2$
*	Multiplicación	$5 * 3 = 15$
/	División (flotante)	$5 / 3 = 1.666...$
//	División entera	$5 // 3 = 1$
%	Módulo (resto)	$5 \% 3 = 2$
**	Potencia	$5 ** 3 = 125$

Cuadro 1: Operadores aritméticos en Python

```

7 resta = a - b
8 multiplicacion = a * b
9 division = a / b          # Divisi n flotante
10 division_entera = a // b  # Divisi n entera
11 modulo = a % b
12 potencia = a ** b
13
14 print(f"Suma: {suma}")
15 print(f"Resta: {resta}")
16 print(f"Multiplicaci n: {multiplicacion}")
17 print(f"Divisi n flotante: {division}")
18 print(f"Divisi n entera: {division_entera}")
19 print(f"M dulo: {modulo}")
20 print(f"Potencia: {potencia}")

```

5.4 Operadores de asignación

Python también ofrece operadores de asignación que combinan una operación con la asignación:

```

1 x = 5
2 x += 3    # Equivale a x = x + 3  -> x = 8
3 x -= 2    # Equivale a x = x - 2  -> x = 6
4 x *= 4    # Equivale a x = x * 4   -> x = 24
5 x /= 3    # Equivale a x = x / 3   -> x = 8
6 x %= 3    # Equivale a x = x % 3   -> x = 2
7 x **= 2   # Equivale a x = x ** 2  -> x = 4

```

6 Programas completos de ejemplo

6.1 Ejemplo 1: Calculadora simple

```

1 # Calculadora de dos n meros
2 print("=== CALCULADORA SIMPLE ===")
3
4 num1 = int(input("Ingresa el primer n mero: "))
5 num2 = int(input("Ingresa el segundo n mero: "))

```

```

6
7 print("\nResultados:")
8 print(f"Suma: {num1} + {num2} = {num1 + num2}")
9 print(f"Resta: {num1} - {num2} = {num1 - num2}")
10 print(f"Multiplicaci n: {num1} * {num2} = {num1 * num2}")
11 print(f"Divisi n exacta: {num1} / {num2} = {num1 / num2}")
12 print(f"Divisi n entera: {num1} // {num2} = {num1 // num2}")
13 print(f"Resto: {num1} % {num2} = {num1 % num2}")
14 print(f"Potencia: {num1} ** {num2} = {num1 ** num2}")

```

6.2 Ejemplo 2: Conversi3n de unidades

```

1 # Convertir segundos a minutos y horas
2 print("== CONVERSION DE TIEMPO ==")
3
4 segundos = int(input("Ingresa los segundos: "))
5
6 minutos = segundos // 60
7 segundos_restantes = segundos % 60
8 horas = minutos // 60
9 minutos_restantes = minutos % 60
10
11 print(f"{segundos} segundos equivalen a:")
12 print(f"{horas} horas, {minutos_restantes} minutos y {
    segundos_restantes} segundos")

```

6.3 Ejemplo 3: Promedio de calificaciones

```

1 # Calcular promedio de tres calificaciones
2 print("== PROMEDIO DE CALIFICACIONES ==")
3
4 cal1 = int(input("Ingresa la calificaci n 1: "))
5 cal2 = int(input("Ingresa la calificaci n 2: "))
6 cal3 = int(input("Ingresa la calificaci n 3: "))
7
8 promedio = (cal1 + cal2 + cal3) / 3
9 promedio_entero = (cal1 + cal2 + cal3) // 3
10
11 print(f"El promedio exacto es: {promedio:.2f}")
12 print(f"El promedio entero es: {promedio_entero}")

```

7 Resumen y recomendaciones

7.1 Puntos clave

- `input()` siempre devuelve una cadena de texto
- `int()` convierte valores a n3meros enteros

- Es necesario usar `int()` con `input()` para operaciones matemáticas
- `print()` muestra información en la consola
- Los enteros en Python soportan operaciones aritméticas básicas
- La división con `/` devuelve flotante, `//` devuelve entero

7.2 Errores comunes a evitar

1. Olvidar convertir el input a entero:

```
1      # Incorrecto
2      num = input("N mero: ")
3      resultado = num + 5  # Error: TypeError
4
5      # Correcto
6      num = int(input("N mero: "))
7      resultado = num + 5
8
```

2. Intentar convertir texto no numérico con `int()`:

```
1      # Esto genera error
2      # numero = int("abc")
3
```

3. Confundir división flotante con división entera:

```
1      print(7 / 2)      # 3.5 (flotante)
2      print(7 // 2)     # 3   (entero)
3
```

8 Conclusión

Los comandos `input()`, `print()` y `int()` son herramientas fundamentales en Python que permiten la interacción con el usuario y el manejo de datos numéricos. Comprender su funcionamiento y saber combinar estos elementos es esencial para desarrollar programas que reciban información del usuario, la procesen y muestren resultados de manera efectiva.

Las operaciones con variables enteras proporcionan la base para realizar cálculos y resolver problemas computacionales, desde los más simples hasta los más complejos. Dominar estos conceptos es el primer paso para convertirse en un programador competente en Python.


```
print ("el area de un circulo")
print ("====el area de un circulo====")

C= 3.14
r= 5
#A= pi*r**2
print (f"el area de el circulo es, A={C*r**2}")
```

```
el area de un circulo
====el area de un circulo====
el area de el circulo es, A=78.5
```


CIENCIAS DE LA COMPUTACIÓN Y PEDAGOGÍA

Tarea 5: Rediseño Curricular

PROPUESTA DE REDISEÑO CURRICULAR

“Synergistic CS Pedagogy:
Reestructuración de la Enseñanza de
Programación Inicial”

Fundamentación:

Conferencia del Dr. Raj Reddy - “*The Future of AI: Doomers vs. Abundance*”
<https://www.youtube.com/watch?v=ydnOSMbyyQo>

Estudiante: [Nombre del Estudiante]

Asignatura: Fundamentos de Programación y Pensamiento Computacional

Fecha de Entrega: 30 de julio de 2026

Profesor: [Nombre del Profesor]

Institución: [Nombre de la Universidad]

Resumen Ejecutivo

Propuesta de rediseño curricular para la enseñanza de programación inicial, basada en el modelo 70/30 (Meta-Aprendizaje / Aprendizaje Justo a Tiempo), alineada con los principios de la conferencia del Dr. Raj Reddy sobre el futuro de la IA y la educación.

Índice

1. Contexto e Importancia Curricular	2
1.1. El Paradigma del Dr. Raj Reddy	2
1.2. Principios Rectores	2
1.3. Justificación del Rediseño	2
2. Objetivo de la Actividad	3
2.1. Objetivos Específicos	3
3. Estructura del Trabajo a Entregar	3
3.1. Distribución Modular del Syllabus (Modelo 70/30)	3
3.1.1. Descripción del Modelo	3
3.1.2. Tabla de Distribución Semanal	4
3.2. Mecanismos de Evaluación Anti-Plagio y Supervisión Humana	4
3.2.1. Modalidades de Evaluación	4
3.2.2. Rúbrica de Evaluación General	5
3.3. Guía Práctica para Docentes	6
3.3.1. Protocolo de Uso de IA en el Aula	6
3.3.2. Directrices Generales para Docentes	6
3.3.3. Recomendaciones para la Implementación	7
4. Formato del Entregable	7
4.1. Justificación Pedagógica (500 palabras)	8
4.1.1. La Metacognición como Eje del Aprendizaje	8
4.1.2. El Aprendizaje Justo a Tiempo como Complemento	8
4.1.3. El Modelo Maharaja y la Evaluación Anti-Plagio	8
4.1.4. Conclusión	8
5. Conclusiones y Recomendaciones	9
5.1. Conclusiones Principales	9
5.2. Recomendaciones para la Implementación	9
5.3. Reflexión Final	9

1 Contexto e Importancia Curricular

La expansión acelerada de las herramientas de Inteligencia Artificial ha transformado radicalmente el panorama de la educación en ciencias de la computación. Los syllabus tradicionales de programación inicial, que basaban gran parte de su evaluación en la memorización sintáctica y la implementación manual de estructuras simples, han quedado obsoletos frente a la capacidad de los asistentes de IA para generar código funcional en segundos.

1.1 El Paradigma del Dr. Raj Reddy

En su conferencia “*The Future of AI: Doomers vs. Abundance*”, el Dr. Raj Reddy plantea que el aula moderna debe adaptarse a un ecosistema donde la IA provee tutoría continua y respuestas bajo demanda (*Just-in-Time*). Reddy argumenta que el verdadero peligro no radica en la IA en sí, sino en la incapacidad de las instituciones educativas para reinventar sus métodos de enseñanza.

1.2 Principios Rectores

Para evitar la pérdida de rigor académico, las facultades de ingeniería deben estructurar currículos que integren armónicamente dos ejes fundamentales:

- **“Aprender a Aprender”** como eje de pensamiento abstracto y desarrollo de habilidades metacognitivas.
- **“Aprender Justo a Tiempo”** como vehículo de agilidad tecnológica y adaptación a herramientas emergentes.

Esta propuesta curricular se fundamenta en la premisa de que el futuro profesional de la computación no será aquel que memorice sintaxis, sino aquel que sepa formular problemas, evaluar soluciones y adaptarse a entornos tecnológicos cambiantes.

1.3 Justificación del Rediseño

El modelo propuesto responde a tres necesidades críticas:

1. **Relevancia Tecnológica:** Preparar a los estudiantes para un mercado laboral donde la IA es una herramienta cotidiana.
2. **Rigor Académico:** Mantener altos estándares de pensamiento computacional y razonamiento lógico.
3. **Autonomía del Estudiante:** Fomentar la capacidad de autoaprendizaje y adaptación continua.

2 Objetivo de la Actividad

El estudiante, en su rol de diseñador curricular y docente, debe proponer un plan de estudios renovado para la asignatura “**Fundamentos de Programación y Pensamiento Computacional**”, adaptado a la era de la Inteligencia Artificial General (AGI).

2.1 Objetivos Específicos

- Diseñar un syllabus que equilibre el desarrollo de habilidades fundamentales con la integración de herramientas de IA.
- Implementar mecanismos de evaluación que garanticen la integridad académica y el desarrollo del pensamiento crítico.
- Proporcionar directrices claras para el uso pedagógico de la IA en el aula.

3 Estructura del Trabajo a Entregar

La propuesta se desarrolla en los siguientes cuatro componentes principales, detallados en las secciones subsiguientes.

3.1 Distribución Modular del Syllabus (Modelo 70/30)

El diseño académico se estructura en 16 semanas, distribuidas en dos bloques complementarios que se desarrollan de manera concurrente a lo largo del semestre.

3.1.1. Descripción del Modelo

- **70 % Meta-Aprendizaje (*Aprender a Aprender*):**
 - Algoritmos abstractos y fundamentos de lógica booleana.
 - Tipos de Datos Abstractos (TDA) y estructuras de datos fundamentales.
 - Análisis de complejidad algorítmica $O(n)$, $O(n^2)$, $O(\log n)$.
 - Arquitectura de computadores: memoria, CPU, y modelo de Von Neumann.
 - Métodos de autodidaxis y técnicas de estudio metacognitivo.
- **30 % Aprendizaje Justo a Tiempo (JIT):**
 - Integración de frameworks modernos (React, Flask, etc.).
 - Librerías avanzadas para análisis de datos y visualización.
 - Prompting estructurado mediante asistentes virtuales de IA (ChatGPT, Copilot, etc.).
 - Desarrollo de proyectos con asistencia de IA.

3.1.2. Tabla de Distribución Semanal

Semana	70 % Meta-Aprendizaje	30 % Aprendizaje JIT
1	Introducción al Pensamiento Computacional	Configuración de entorno de desarrollo con IA
2	Lógica Booleana y Álgebra de Conjuntos	Introducción al prompting efectivo
3	Algoritmos: Definición y Propiedades	Uso de ChatGPT para depuración de código
4	Tipos de Datos Abstractos: Pilas y Colas	Implementación asistida de TDA
5	Análisis de Complejidad $O(n)$	Optimización con sugerencias de IA
6	Estructuras de Datos: Listas y Arreglos	Uso de bibliotecas estándar con IA
7	Arquitectura de Computadores	Desarrollo de proyectos con IA
8	Examen Parcial (Sin IA)	Revisión y corrección asistida
9	Recursión y Algoritmos Divide y Vencerás	Implementación recursiva con IA
10	Ordenamiento y Búsqueda: Algoritmos Clásicos	Análisis de eficiencia con herramientas de IA
11	Árboles y Grafos: Fundamentos	Visualización de grafos con IA
12	Técnicas de Autodidaxis y Metacognición	Uso de IA para autoaprendizaje
13	Métodos de Optimización y Complejidad	IA para optimización de código
14	Proyecto Final: Diseño Conceptual	Desarrollo del proyecto con IA
15	Revisión y Preparación para Exámenes	Evaluación de proyectos con IA
16	Examen Final (Con defensa oral)	Presentación de proyectos integradores

3.2 Mecanismos de Evaluación Anti-Plagio y Supervisión Humana

Basados en el “Modelo Maharaja” propuesto por el Dr. Reddy, donde el estudiante debe defender su criterio de supervisión, se establecen los siguientes mecanismos de evaluación:

3.2.1. Modalidades de Evaluación

1. Defensa Oral de Código (40 % de la nota final):

- El estudiante presenta un proyecto desarrollado durante el semestre.
- Debe explicar cada decisión de diseño, justificando la elección de algoritmos y estructuras de datos.
- El comité evaluador formula preguntas para verificar la comprensión profunda.
- **Criterio de evaluación:** Claridad expositiva, profundidad conceptual, capacidad de argumentación.

2. Análisis de Código Defectuoso Generado por IA (30 %):

- Se proporciona código generado por IA que contiene errores lógicos, de eficiencia o de seguridad.
- El estudiante debe identificar, diagnosticar y corregir los defectos.
- Debe presentar un informe técnico que explique la naturaleza de cada error.
- **Criterio de evaluación:** Precisión en la detección, calidad de la solución, profundidad del análisis.

3. Refactorización en Tiempo Real (30 %):

- El estudiante recibe un código funcional pero ineficiente.
- En un entorno supervisado (sin acceso a IA), debe refactorizarlo para mejorar su eficiencia y legibilidad.
- Debe explicar en voz alta cada paso del proceso.
- **Criterio de evaluación:** Eficiencia de la solución, manejo del tiempo, capacidad de comunicación.

3.2.2. Rúbrica de Evaluación General

Criterio	Excelente (90-100 %)	Satisfactorio (70-89 %)	Necesita Mejora (<70 %)
Dominio Conceptual	Demuestra comprensión profunda y aplica conceptos con creatividad	Comprende los conceptos fundamentales y los aplica correctamente	Muestra lagunas conceptuales y aplica incorrectamente los conceptos
Razonamiento Lógico	Identifica y resuelve problemas complejos con elegancia	Resuelve problemas estándar sin dificultad	Tiene dificultades para resolver problemas básicos

Continúa en la siguiente página

Criterio	Excelente (90-100 %)	Satisfactorio (70-89 %)	Necesita Mejora (¡70 %)
Análisis de Complejidad	Realiza análisis exhaustivo de complejidad temporal y espacial	Realiza análisis básico de complejidad	No realiza análisis de complejidad o lo hace incorrectamente
Comunicación y Defensa	Explica con claridad y profundidad, respondiendo preguntas con soltura	Se expresa con claridad, pero con limitaciones en preguntas complejas	Dificultades para expresar sus ideas o defender sus decisiones

3.3 Guía Práctica para Docentes

3.3.1. Protocolo de Uso de IA en el Aula

Fase del Aprendizaje	Uso de IA Permitido	Justificación Pedagógica
Evaluación Sumativa	No permitido	Garantiza la medición del desarrollo de habilidades fundamentales
Evaluación Formativa	Permitido	Fomenta el aprendizaje autónomo y la autoevaluación
Proyectos Integradores	Permitido con supervisión	Desarrolla habilidades de colaboración hombre-máquina
Prácticas de Laboratorio	Permitido	Acelera el aprendizaje de herramientas y frameworks
Talleres de Pensamiento Crítico	No permitido	Desarrolla razonamiento independiente y solución de problemas
Desarrollo de Algoritmos	No permitido (inicial) / Permitido (avanzado)	Permite el desarrollo de habilidades fundamentales antes de acelerar el proceso

Cuadro 3: Protocolo de Uso de IA en el Aula

3.3.2. Directrices Generales para Docentes

1. Fase 1: Introducción a Conceptos Fundamentales (Semanas 1-7)

- **IA Permitida:** No en evaluación, sí en tareas formativas.

- **Objetivo:** Desarrollar el pensamiento computacional sin dependencia tecnológica.
 - **Actividades:** Ejercicios de lógica, pseudocódigo, diagramas de flujo.
2. **Fase 2: Desarrollo de Herramientas de Autodidaxis (Semanas 9-13)**
- **IA Permitida:** Sí, como asistente de aprendizaje.
 - **Objetivo:** Enseñar a los estudiantes a utilizar la IA para el autoaprendizaje.
 - **Actividades:** Proyectos de investigación con IA, resolución de problemas con asistencia.
3. **Fase 3: Integración y Síntesis (Semanas 14-16)**
- **IA Permitida:** Sí, en desarrollo de proyectos con supervisión.
 - **Objetivo:** Integrar las habilidades fundamentales con herramientas avanzadas.
 - **Actividades:** Proyectos finales, presentaciones, defensas orales.

3.3.3. Recomendaciones para la Implementación

- **Capacitación Docente:** Los profesores deben recibir formación en el uso pedagógico de la IA.
- **Infraestructura Tecnológica:** Garantizar acceso a herramientas de IA para todos los estudiantes.
- **Ética y Plagio:** Establecer políticas claras sobre el uso de IA y la propiedad intelectual.
- **Evaluación Continua:** Implementar mecanismos de retroalimentación para ajustar el modelo según las necesidades del grupo.

4 Formato del Entregable

El presente documento constituye una propuesta académica formal que incluye:

1. **Syllabus Completo:** Distribución modular detallada en tablas.
2. **Rubros de Evaluación:** Criterios y ponderaciones definidos.
3. **Justificación Pedagógica:** Fundamentación teórica de 500 palabras (presentada a continuación).

4.1 Justificación Pedagógica (500 palabras)

La propuesta curricular aquí presentada se fundamenta en los principios expuestos por el Dr. Raj Reddy en su conferencia “*The Future of AI: Doomers vs. Abundance*”. Reddy plantea que el advenimiento de la Inteligencia Artificial no debe ser visto como una amenaza, sino como una oportunidad para redefinir la educación. El modelo 70/30 propuesto responde a esta visión, equilibrando el desarrollo de habilidades fundamentales con la integración de herramientas tecnológicas avanzadas.

4.1.1. La Metacognición como Eje del Aprendizaje

El 70 % dedicado al *Meta-Aprendizaje* se basa en la premisa de que el pensamiento computacional es una habilidad transferible que trasciende cualquier lenguaje o herramienta específica. Algoritmos abstractos, lógica booleana, tipos de datos abstractos y análisis de complejidad son conceptos que no cambiarán significativamente con el tiempo, a diferencia de las herramientas tecnológicas. Este enfoque prepara a los estudiantes para enfrentar problemas complejos y adaptarse a entornos cambiantes.

4.1.2. El Aprendizaje Justo a Tiempo como Complemento

El 30 % dedicado al *Aprendizaje Justo a Tiempo* reconoce que la velocidad de cambio tecnológico hace imposible enseñar todas las herramientas que un estudiante encontrará en su carrera profesional. En lugar de intentar cubrir un espectro amplio de tecnologías, este modelo enseña a los estudiantes a aprender de manera autónoma, utilizando la IA como catalizador. Como señala Reddy, la capacidad de encontrar y utilizar información bajo demanda es una habilidad esencial en la era de la AGI.

4.1.3. El Modelo Maharaja y la Evaluación Anti-Plagio

Los mecanismos de evaluación propuestos, inspirados en el *Modelo Maharaja* de Reddy, garantizan que el estudiante no pueda eludir el desarrollo de habilidades fundamentales mediante el uso de IA. Las defensas orales y el análisis de código defectuoso evalúan la comprensión profunda y la capacidad crítica, habilidades que no se pueden externalizar a una máquina. La refactorización en tiempo real, por su parte, mide la capacidad de razonamiento y adaptación del estudiante.

4.1.4. Conclusión

Esta propuesta curricular no solo actualiza los contenidos de la enseñanza de programación inicial, sino que transforma el rol del docente de transmisor de conocimiento a facilitador del aprendizaje. Siguiendo las ideas de Reddy, el aula se convierte en un espacio de colaboración entre humanos y máquinas, donde la IA amplifica las capacidades humanas en lugar de reemplazarlas.

5 Conclusiones y Recomendaciones

El rediseño curricular propuesto para la asignatura “*Fundamentos de Programación y Pensamiento Computacional*” representa un paso necesario hacia una educación más relevante, rigurosa y adaptada a los desafíos del siglo XXI.

5.1 Conclusiones Principales

1. **Equilibrio entre Fundamentos y Tecnología:** El modelo 70/30 permite mantener el rigor académico mientras se integran herramientas modernas.
2. **Evaluación Anti-Plagio:** Los mecanismos de defensa oral y análisis de código aseguran la integridad académica.
3. **Flexibilidad Docente:** La guía práctica proporciona directrices claras sin ser restrictiva.
4. **Preparación para el Futuro:** El enfoque en el autoaprendizaje prepara a los estudiantes para un mercado laboral en constante cambio.

5.2 Recomendaciones para la Implementación

- **Fase Piloto:** Implementar el modelo en un grupo reducido antes de escalarlo.
- **Retroalimentación Continua:** Recoger y analizar la opinión de estudiantes y docentes.
- **Actualización Constante:** Revisar el syllabus cada semestre para incorporar nuevas herramientas y metodologías.
- **Colaboración Interdisciplinaria:** Involucrar a expertos en pedagogía y tecnología para mejorar el diseño.

5.3 Reflexión Final

Como bien señala el Dr. Raj Reddy, el futuro de la educación no se trata de competir con la IA, sino de aprovechar su potencial para potenciar el aprendizaje humano. Esta propuesta curricular es un primer paso hacia esa visión, invitando a docentes y estudiantes a embarcarse en un viaje de descubrimiento y transformación.

Referencias

1. Reddy, R. (2024). *The Future of AI: Doomers vs. Abundance*. Conferencia magistral. Disponible en: <https://www.youtube.com/watch?v=ydn0SMbbyyQo>
2. Wing, J. M. (2006). Computational Thinking. *Communications of the ACM*, 49(3), 33-35.

3. Papert, S. (1980). *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books.
4. Bloom, B. S. (1956). *Taxonomy of Educational Objectives*. Longmans, Green and Co.
5. Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.
6. Hazzan, O., & Lapidot, T. (2004). *Constructivist Computer Science Education*. ACM SIGCSE Bulletin, 36(1), 266-270.

Anexos

Anexo A: Ejemplo de Rúbrica Detallada para Defensa Oral

Criterio	Excelente (9-10)	Bueno (7-8)	Satisfactorio (5-6)	Necesita Mejora (¡5)
Profundidad Conceptual	Demuestra dominio excepcional de conceptos	Comprende bien los conceptos fundamentales	Comprende algunos conceptos básicos	Muestra lagunas importantes
Comunicación y Claridad	Se expresa con excepcional claridad y precisión	Se expresa con claridad	Se expresa de manera comprensible	Dificultades significativas de comunicación
Capacidad de Respuesta	Responde preguntas con precisión y profundidad	Responde preguntas correctamente	Responde preguntas básicas	Dificultades para responder preguntas
Organización y Estructura	Presenta ideas de manera lógica y coherente	Presentación bien estructurada	Presentación aceptable	Desorganización en la presentación

Anexo B: Ejemplo de Ejercicio de Refactorización

Enunciado: Se proporciona el siguiente código en Python que calcula la suma de los primeros n números enteros de manera ineficiente. Refactorízelo para mejorar su complejidad temporal y espacial, manteniendo la misma funcionalidad.

```
def suma_numeros(n):
    suma = 0
    for i in range(1, n+1):
```

```
    for j in range(1, i+1):
        suma += 1
    return suma
```

Requerimientos:

- Identificar y explicar los problemas de eficiencia del código original.
- Proponer una implementación alternativa que reduzca la complejidad de $O(n^2)$ a $O(1)$.
- Justificar por qué la nueva implementación es superior.

Anexo C: Ejemplo de Prompting Estructurado para IA

Ejemplo de Prompt para Asistente de IA:

Actúa como un tutor experto en programación. Estoy aprendiendo estructuras de datos y necesito entender el concepto de pilas.

Por favor:

1. Define qué es una pila y sus operaciones fundamentales (push, pop, top).
2. Proporciona un ejemplo en Python de implementación de una pila.
3. Explica un caso de uso práctico en el desarrollo de software.
4. Plantea un ejercicio para que yo resuelva sobre el uso de pilas.

```
# Ejemplo de tipos numericos
a = 10 # Entero
b = 3.14 # Punto flotante
c = a + b
print(c) # Salida: 13.14
```

13.14

```
# Ejemplo de cadena de texto
nombre = "juan"
saludo = "Hola,"+ nombre
print(saludo) # Salida: Hola, Juan
```

Hola,juan

```
print (f"juan recibe despues de decir {saludo} un vuelto de {c}")
```

juan recibe despues de decir Hola,juan un vuelto de 13.14

```
import math
print(math.pi)
```

3.141592653589793

```
import math
print(f"{math.pi:.10f}")
```

3.1415926536
3.1415926536

```
print("====lea con cuidado lo que se pide====")
a= input ("ingresa el numero entero deseado de cifras significativas: ")
a=int(a)
import math
print (f"el numero pi con {a} cifras decimales es igual a: {math.pi:{a}f}")
b=4
c=a+b
print(c)
```

```
====lea con cuidado lo que se pide====
ingresa el numero entero deseado de cifras significativas: 10
el numero pi con 10 cifras decimales es igual a: 3.141593
14
```

```
# Python como calculadora
print("=== Operaciones Basicas ===")
print(f"Suma: 2 + 3 = {2 + 3}")
print(f"Resta: 10 - 4 = {10 - 4}")
print(f"Multiplicacion: 6 * 7 = {6 * 7}")
print(f"Division: 15 / 3 = {15 / 3}")
print(f"Potencia: 2 ** 8 = {2 ** 8}")
print(f"Modulo (Residuo): 17 % 5 = {17 % 5}")
```

```
=== Operaciones Basicas ===
Suma: 2 + 3 = 5
Resta: 10 - 4 = 6
Multiplicacion: 6 * 7 = 42
Division: 15 / 3 = 5.0
Potencia: 2 ** 8 = 256
Modulo (Residuo): 17 % 5 = 2
```

```
def saludar (nombre):
    return f"hola,{nombre}! eres una chica formidable"
# llamada a la funcion
nombre = input("ingresa tu nombre:")
mensaje= saludar (nombre)
print(mensaje) # hola, "nombre"!
```

```
ingresa tu nombre:andrea
hola,andrea! eres una chica formidable
```

```
def andrea_suma(a,b):  
    return f"Esta es la suma {a} y {b} cuyo resultado es:{a+b}"  
  
andrea_suma(2,1)
```

```
'Esta es la suma 2 y 1 cuyo resultado es:3'
```

```
def andrea_resta (c,d):  
    return f"esta es la resta del {c} y del {d} cuyo resultado es: {c-d}"  
  
andrea_resta(5,2)
```

```
'esta es la resta del 5 y del 2 cuyo resultado es: 3'
```

```
def Multiplicacion (e,f):  
    return f"esta es la multiplicacion del {e} y del {f} cuyo resultado es: {e*f}"  
  
Multiplicacion(5,2)
```

```
'esta es la multiplicacion del 5 y del 2 cuyo resultado es: 10'
```

```
def division (h,g):  
    return f"esta es la division del {h} y del {g} cuyo resultado es: {h/g}"  
  
division(10,2)
```

```
'esta es la division del 10 y del 2 cuyo resultado es: 5.0'
```

```
def potencia (i,j):  
    return f"esta es la potencia del {i} y del {j} cuyo resultado es: {i**j}"  
  
potencia(2,3)
```

```
'esta es la potencia del 2 y del 3 cuyo resultado es: 8'
```

```
def residuo(k,l):  
    return f"este es el residuo de la division entre {k} y {l} cuyo resltado es {k%l}"  
  
residuo (20,3)
```

```
'este es el residuo de la division entre 20 y 3 cuyo resultado es 2'
```

```
# funcion para hallar el area de un cuadrado, conociendo su lado  
def area_cuadrado(lado):  
    return f"el area del cuadrado de lado {lado} es {lado**2} unidades cuadradas"  
  
area_cuadrado(5)
```

```
'el area del cuadrado de lado 5 es 25 unidades cuadradas '
```

```
# funcion para hallar el area de un rectangulo, conociendo su base y altura  
def area_rectangulo(base, altura):  
    return f"el area del rectangulo de base {base} y altura {altura} es {base*altura} unidades cuadradas"  
  
area_rectangulo(5,3)
```

```
'el area del rectangulo de base 5 y altura 3 es 15 unidades cuadradas'
```



```
def andrea_raiz(x):
    y=(3+x**(1/2))/((x-1)**(1/2)+3)
    return y

andrea_raiz(1)
```

```
1.3333333333333333
```

```
def andrea_dos(x,n):
    y=(3+x**(1/2))/((x-1)**(1/2)+3)
    w=f"{y:.{n}f}"
    return w

andrea_dos(1,4)
```

```
'1.3333'
```

```
def andrea_dos(x,n):
    y=(3+x**(0.5))/((x-1)**(0.5)+3)
    w=f"{y:.{n}f}"
    return w

andrea_dos(1,4)
```

```
'1.3333'
```

```
import math

def andrea_tres(x,n):
    y=(3+x**(1/2))/((x-1)**(1/2)+3)
    w=f"{y:.{n}f}"
    return w

andrea_tres(1,4)

math.sqrt(2)
```

```
1.4142135623730951
```

```
import math

def andrea_tres(x,n):
    y=(3+x**(1/2))/((x-1)**(1/2)+3)
    w=f"{y:.{n}f}"
    z=math.sqrt(2)
    m=f"{z:.{n}f}"

    return f"el valor de la funcion es: {w} y el valor de la raiz de 2 es: {m}"

andrea_tres(1,4)
#math.sqrt(2)
# print (f"{andrea_tres(1,10)}y {math.sqrt(2)}")
```

```
'el valor de la funcion es: 1.3333 y el valor de la raiz de 2 es: 1.4142'
```

```
import math
print(2**(1/3))

pi= math.pi
print(pi)
print(f"{pi:.5f}")

e=math.e
print(e)
print(f"{e:.5f}")
```

```
1.2599210498948732
3.141592653589793
3.14159
2.718281828459045
2.71828
```



```
#raiz cuadrada de 2

z=f"{2**(1/2):.2f}"
print(z)

#raiz cuadrada de 3

k=f"{3**(1/2):.2f}"
print(k)

#euler
e=math.e
#print(e)
print(f"{e:.2f}")

#pi
pi= math.pi
print(pi)
print(f"{pi:.2f}")
```

```
1.41
1.73
2.72
3.141592653589793
3.14
```

```
def andrea(d):
    x=2**(1/2)
    a=f"{x:.{d}f}"
    y=3**(1/2)
    b=f"{y:.{d}f}"
    z=math.e
    c=f"{z:.{d}f}"
    k=math.pi
    e=f"{k:.{d}f}"
    return f"el valor de la raiz cuadrada de a es: {a}, el valor de la raiz cuadrada de b es: {b}, el valor de e es: {c} y el valor de pi es: {e}"

andrea(4)
```

```
'el valor de la raiz cuadrada de a es: 1.4142, el valor de la raiz cuadrada de b es: 1.7321, el valor de e es: 2.7183 y el valor de pi es: 3.1416'
```